Code Coverage Report — Pizza Order

Lab 6: Code and Branch Coverage Analysis | Application: pizza_order.py (SETAPESU26/L6_L7_24_pizza_order, Lab 6)

Student: Akshat Agarwal, PES1UG24AM024 | Repository: github.com/Akshat1686/L6_pizza_order_024

Environment verified: Python 3.12.3 / pytest 9.2.0 / coverage 7.17.0 on Linux, and Python 3.12.10 / pytest 9.2.0 on Akshat's Windows terminal (screenshots in this report). The application file pizza_order.py was not changed; only test files were added.

1. What code (line) coverage means

Code coverage, also called line coverage, asks one question about each executable line of source code: did it run at least once during testing? It does not ask whether every decision went both ways — that is branch coverage, covered in the companion report. A line can therefore be 'covered' even if only one side of the if statement guarding it was ever tested. That is why the lab requires both reports, and why a 100% line score is a necessary, but not sufficient, proof of thorough testing.

The lab's pass mark is at least 90% code coverage. This suite exceeds it at 100%.

2. Application under test

pizza_order.py has three functions and 34 executable statements. No function was edited for this lab.

- pizza_price(size, toppings, is_cheese_burst) — base prices small 200, medium 350, large 500; toppings cost 40 each up to 3, then 40 each for the first 3 plus 30 for each extra topping; cheese burst adds 100, but is rejected for small. Unknown sizes and negative toppings raise ValueError.
- order_discount(order_total, day, is_student) — orders under 300 get no discount at all (an early return, so even a student gets 0); Tuesday gives half the total; Saturday/Sunday give 0; any other day gives a tenth; a student adds 50 on top of whichever day discount applied.
- delivery_status(minutes_taken, distance_km) — over 10 km is 'not deliverable'; up to 30 minutes is 'on time'; up to 45 minutes is 'late'; anything longer is 'free pizza'.

3. Starting point: the starter tests

The template's test_pizza_order.py contained only two tests: a plain medium pizza and an on-time delivery. Measured before any new test was written, pizza_order.py was at 41% line coverage — 20 of its 34 statements never ran, namely lines 6, 8, 11, 15–17, 22–32, 37 and 40–42. In other words, every error case, the >3-toppings tier, cheese burst, the whole discount function, and the late / free-pizza delivery paths were completely untested.

4. The code-coverage test suite

File: CodeCoverageTest_024_pizza_order.py (024 = PES1UG24AM024). 20 tests, grouped by function. Each test below names its input, the expected result, and the lines it exists to cover; line numbers refer to pizza_order.py.

4.1 pizza_price — 9 tests

#	Test	Input -> Expected	What it covers
1	test_plain_medium_pizza	medium, 0, False -> 350	Base price; toppings else-branch (line 13); return (18)
2	test_small_two_toppings_no_cheese	small, 2, False -> 280	200 + 2x40; toppings else-branch
3	test_large_three_toppings_no_cheese	large, 3, False -> 620	Top of the <=3 tier: 500 + 3x40
4	test_large_five_toppings_no_cheese	large, 5, False -> 680	>3 tier (line 11): 500 + 120 + 2x30
5	test_medium_cheese_burst	medium, 2, True -> 530	Cheese-burst addition (line 17): 350 + 80 + 100
6	test_large_cheese_burst_no_toppings	large, 0, True -> 600	Cheese burst with no toppings: 500 + 100
7	test_unknown_size_raises	extra-large, 1, False -> ValueError	Unknown-size guard (line 6)
8	test_negative_toppings_raises	medium, -1, False -> ValueError	Negative-toppings guard (line 8)
9	test_cheese_burst_small_raises	small, 1, True -> ValueError	Cheese-burst-on-small guard (line 16)

4.2 order_discount — 7 tests

#	Test	Input -> Expected	What it covers
10	test_discount_below_minimum_is_zero	250, tuesday, False -> 0	Early return (line 23): under 300, no discount even on Tuesday
11	test_discount_tuesday_half	400, tuesday, False -> 200	Tuesday half (line 25)
12	test_discount_tuesday_half_plus_student	400, tuesday, True -> 250	Tuesday half + student +50 (lines 25, 31)
13	test_discount_weekend_is_zero	400, saturday, False -> 0	Weekend discount 0 (line 27)
14	test_discount_weekend_student_only	500, sunday, True -> 50	Weekend 0, but student +50 still applies
15	test_discount_weekday_te	400, monday, False -> 40	Other-day tenth (line 29)

	nth		
16	test_discount_weekday_tenth_plus_student	400, wednesday, True -> 90	Other-day tenth + student +50

4.3 delivery_status — 4 tests

#	Test	Input -> Expected	What it covers
17	test_delivery_not_deliverable_over_10km	20 min, 11 km -> not deliverable	Distance guard (line 37)
18	test_delivery_on_time	20 min, 3 km -> on time	Up to 30 minutes (line 39)
19	test_delivery_late	40 min, 5 km -> late	Up to 45 minutes (line 41)
20	test_delivery_free_pizza	60 min, 5 km -> free pizza	Over 45 minutes (line 42)

5. Results

Commands run (from the folder containing pizza_order.py):

```
coverage run -m pytest CodeCoverageTest_024_pizza_order.py -v

coverage report -m
```

File	Statements	Missed	Coverage	Missing
CodeCoverageTest_024_pizza_order.py	45	0	100%	—
pizza_order.py	34	0	100%	—
TOTAL	79	0	100%	—

All 20 tests passed, in 0.04 s on the Linux verification run and in Akshat's own Windows run (Python 3.12.10, pytest 9.2.0). The single warning in the Windows run is an unrelated anyio/pytest deprecation notice (CallSpec2 renamed to CallSpec, to be removed in pytest 10); it is not a test failure and does not affect coverage. The lab requirement (>=90%) is met with 10 points of headroom, and no line of pizza_order.py is left unexecuted.

6. Screenshots (Akshat's Windows terminal)

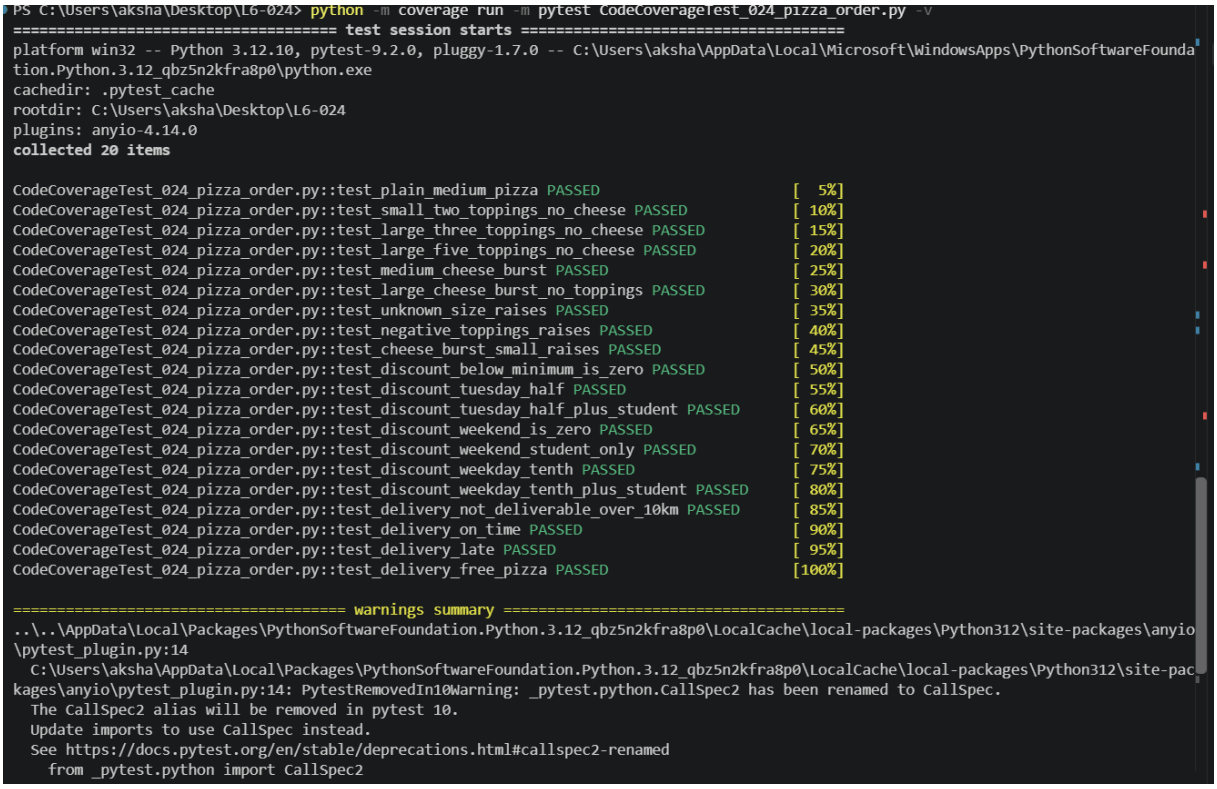


Figure 1 — Code-coverage test run: 20 tests, all PASSED.

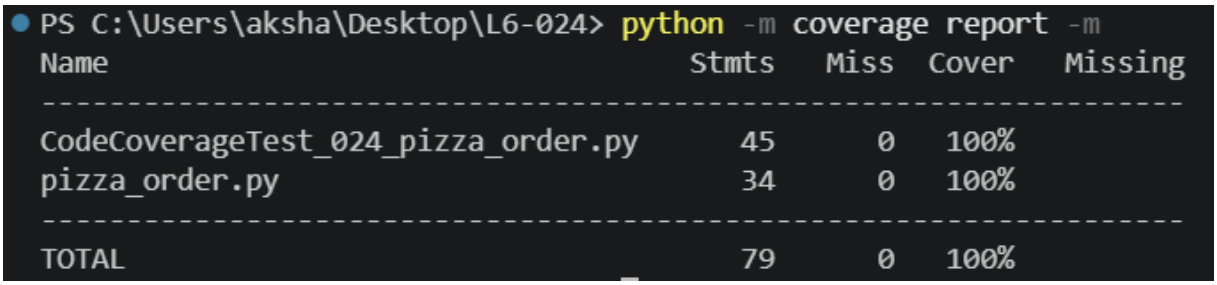


Figure 2 — Code-coverage report: pizza_order.py 34 statements, 0 missed, 100%.

7. Interpretation and limits

- What 100% line coverage proves: every statement, including every error raise and every return, executes under at least one test. The starter suite's blind spots — discounts, errors, cheese burst — are gone.
- What it does not prove: that every decision was tested both ways. For example, a test that only ever ordered cheese-burst pizzas would still execute the '+100' line. That gap is exactly what branch coverage closes, and it is why the companion Branch Coverage Report exists — and why, honestly, this code suite already reaches 100% branch as well, because its tests happen to include both outcomes of the cheese-burst and student flags.

- What it does not prove either: correctness for inputs outside the tested set (other sizes are rejected by design, but untested topping counts above 5 or totals above 500 follow the same arithmetic and were not enumerated). Coverage measures execution, not a proof of correctness.

Conclusion: the code-coverage deliverable is complete — 20 tests, 100% line coverage, all tests passing, with terminal screenshots as evidence.